

STA2053 Special topics in applied statistics

Monica Alexander

Week 4: More Bayes and MCMC

Today

- ▶ Overview of class of methods to generate samples from posterior distributions
- ▶ Introduction to Stan, the platform we will use (through R) to do Bayesian statistical modeling

Reading

- ▶ Hoff Chapters 5,6
- ▶ BDA Chapters 10-13
- ▶ More details on HMC:
 - ▶ Neal, MCMC using Hamiltonian dynamics:
<https://arxiv.org/pdf/1206.1901.pdf>
 - ▶ Betancourt, A Conceptual Introduction to Hamiltonian Monte Carlo: <https://arxiv.org/abs/1701.02434>
- ▶ Other stuff:
 - ▶ Robert and Casella, “A Short History of Markov Chain Monte Carlo: Subjective Recollections from Incomplete Data”, Statistical Science, 2011, 26(1):102-115.
 - ▶ Stanislav Ulam’s autobiography, ‘Adventures of a Mathematician’, is a great read
 - ▶ Arianna Rosenbluth’s obituary

Recap

- ▶ Posterior is proportional to the likelihood times the prior
- ▶ The prior encompasses knowledge to date (before data)
 - ▶ Prior setting involves choice
 - ▶ Pragmatic approach is to choose weakly informative prior
- ▶ Knowledge is then updated based on data collected
- ▶ Bayesian inference revolves around inference based on the posterior

Recap

Bayes rule for more than one parameter

$$p(\theta|\mathbf{y}) = \frac{p(\mathbf{y}|\theta)p(\theta)}{p(\mathbf{y})}$$

- ▶ $p(\mathbf{y}) = \int_{\theta'} p(\mathbf{y}|\theta')p(\theta')d\theta'$
- ▶ The marginal posterior for just one parameter is given by $p(\theta_1|\mathbf{y}) = \int_{\theta'_2} \cdots \int_{\theta'_p} p(\theta_1, \theta'_2, \dots, \theta'_p|\mathbf{y})d\theta'_2 \cdots d\theta'_p$.

Recap

- ▶ Problem: often don't have a closed form solution for posterior
- ▶ Solution: We can make inference about any random variable θ , using a sample from its probability distribution. This is called a Monte Carlo (MC) approximation.
- ▶ If we are able to obtain a sample from posterior $p(\theta|\mathbf{y})$ then
 - ▶ for each parameter we have a sample of its marginal
e.g. $\theta_1^{(1)}, \dots, \theta_1^{(S)} \sim p(\theta_1|y)$.
 - ▶ we can report any summary we'd like, e.g. posterior mean (sample mean), posterior median or other percentiles (sample percentiles).

Marginalization and sampling

- ▶ Joint distribution of parameters

$$p(\theta_1, \theta_2 \mid y) \propto p(y \mid \theta_1, \theta_2)p(\theta_1, \theta_2)$$

- ▶ Marginalization

$$p(\theta_1 \mid y) = \int p(\theta_1, \theta_2 \mid y) d\theta_2$$

$p(\theta_1 \mid y)$ is a marginal distribution

- ▶ Monte Carlo approximation

$$p(\theta_1 \mid y) \approx \frac{1}{S} \sum_{s=1}^S p(\theta_1, \theta_2^{(s)} \mid y),$$

where $\theta_2^{(s)}$ are draws from $p(\theta_2 \mid y)$

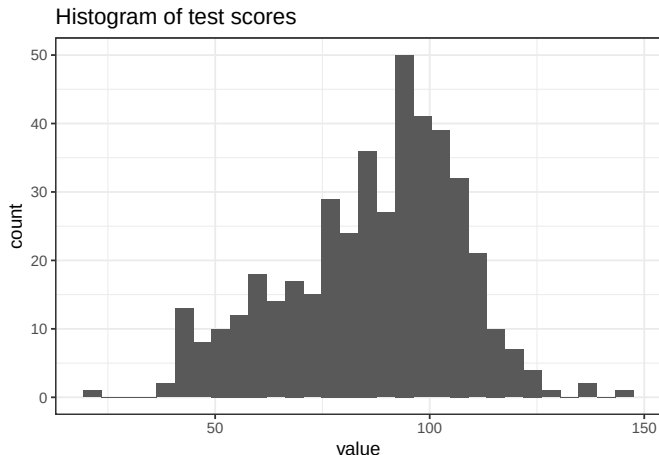
A note on simulation-based inference in general

- ▶ We are going to be talking about and using samples to make inferences a lot, because it's usually the most tractable way of making inferences from the posterior distribution
- ▶ But simulation techniques are super useful more generally
- ▶ Simulate datasets, run models, make sure you get back what you put in
- ▶ Simulate to understand sampling behavior

Example from last week

Outcome of interest: cognitive tests scores for 3-4 year old kids.
Denote the unknown population mean test score by μ , and
observed test score by y_i for kid i , with $i = 1, \dots, n$.

Goal: estimate μ .



Example: kid's test scores

We assume a Normal likelihood

$$Y_i|\mu, \sigma^2 \sim N(\mu, \sigma^2)$$

- ▶ If we put a joint prior $p(\mu, \sigma)$ on the parameters, Bayes' rule tells us how to get the joint posterior distribution:

$$p(\mu, \sigma|\mathbf{y}) = \frac{p(\mathbf{y}|\mu, \sigma)p(\mu, \sigma)}{p(\mathbf{y})}$$

- ▶ And if inference about μ is our goal, we can get the marginal posterior distribution

$$p(\mu|\mathbf{y}) = \int_{\sigma} p(\mu, \sigma|\mathbf{y})d\sigma$$

- ▶ Even with this fairly simple set up (we're not even modeling μ yet!!), it is hard to sample from the joint posterior $p(\mu, \sigma|\mathbf{y})$ directly

Where are we at

- ▶ Bayesian inference revolves around inference based on the posterior
- ▶ Posterior usually hard to write down in closed form
- ▶ But as long as we can get a set of samples from posterior, we can do inference

Where are we going

- ▶ How do we get samples from posterior?
- ▶ How do we run models in R?
- ▶ How do we check models?

Gibbs Sampling

Intuition

- ▶ Pretend for a moment we knew the value of σ in the Normal case. Then to get posterior samples for μ , the problem reduces to one parameter: we would be sampling from the full conditional distribution for μ , $p(\mu|\mathbf{y}, \sigma)$. It is often the case that this is easy to do!
- ▶ A similar strategy could be employed to sample from the full conditional posterior distribution for σ
- ▶ The Gibbs Sampler is an iterative algorithm that sequentially samples parameters from their full conditional distributions, constructing a dependent sequence of parameter values whose distribution converges to the target joint posterior distribution

Gibbs Sampling

Instead of trying to sample directly from $p(\mu, \sigma|\mathbf{y})$, sample parameters sequentially from their **full conditional** distributions (conditioning on data as well as all other model parameters).

Given starting values $\mu^{(1)}$ and $\sigma^{(1)}$, draw samples $s = 1, 2, \dots$ some large number as follows:

1. sample $\mu^{(s+1)}$ from $p(\mu|\mathbf{y}, \sigma^{(s)})$
2. sample $\sigma^{(s+1)}$ from $p(\sigma|\mathbf{y}, \mu^{(s+1)})$

Gibbs Sampling for μ and σ when $y_i \sim N(\mu, \sigma^2)$

In full conditional distributions, data and all other model parameters are assumed known. So let's figure out what they are by considering the following settings:

- ▶ Obtain $p(\mu|\mathbf{y}, \sigma^{(s)})$, in other words, obtain the posterior for μ when assuming that σ is known.
- ▶ Obtain $p(\sigma|\mathbf{y}, \mu^{(s)})$, in other words, obtain the posterior for σ when assuming that μ is known.

Estimating mean test score

Assuming σ is known

It turns out that with a normal prior on μ

$$\mu \sim N(\mu_0, \sigma_{\mu_0}^2)$$

and likelihood function

$$p(\mathbf{y}|\mu, \sigma^2) = \prod_{i=1}^n p(y_i|\mu) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi}\sigma} \exp\left(\frac{-1}{2\sigma^2} (y_i - \mu)^2\right)$$

The posterior is normal too:

$$\mu|\mathbf{y}, \sigma^2 \sim N\left(\frac{\mu_0/\sigma_{\mu_0}^2 + n \cdot \bar{y}/\sigma^2}{1/\sigma_{\mu_0}^2 + n/\sigma^2}, \frac{1}{1/\sigma_{\mu_0}^2 + n/\sigma^2}\right)$$

How did the normal posterior come about?

$$p(\mu|\mathbf{y}, \sigma^2) \propto p(\mu)p(\mathbf{y}|\mu, \sigma^2) \quad (\text{Bayes' rule})$$

$$\propto \exp\left(\frac{-1}{2\sigma_{\mu 0}^2}(\mu - \mu_0)^2\right) \cdot \exp\left(\frac{-1}{2\sigma^2} \sum_{i=1}^n (y_i - \mu)^2\right) \propto \exp\left(-\frac{1}{2}f(\mu)\right)$$

$$\text{where } f(\mu) = 1/\sigma_{\mu 0}^2 (\mu - \mu_0)^2 + \sum_{i=1}^n (y_i - \mu)^2$$

$$\propto 1/\sigma_{\mu 0}^2 \exp\left(-\frac{1}{2\sigma_{\mu 0}^2}(\mu - \mu_0)^2\right)$$

$$\propto (1/\sigma_{\mu 0}^2) \exp\left(-\frac{1}{2\sigma_{\mu 0}^2}(\mu - \mu_0)^2\right)$$

$$\propto (1/\sigma_{\mu 0}^2) \exp\left(-\frac{1}{2\sigma_{\mu 0}^2}(\mu - \mu_0)^2\right)$$

Remember that if $Z \sim N(m, s^2)$, $p(z) \propto \exp(-1/2 \cdot 1/s^2(z - m)^2)$

Thus

$$\mu|\mathbf{y}, \sigma^2 \sim N\left(\frac{\mu_0/\sigma_{\mu 0}^2 + n \cdot \bar{y}/\sigma^2}{(1/\sigma_{\mu 0}^2 + n/\sigma^2)}, (1/\sigma_{\mu 0}^2 + n/\sigma^2)^{-1}\right)$$

What's the interpretation here?

$$\mu|\mathbf{y}, \sigma^2 \sim N\left(\frac{\mu_0/\sigma_{\mu 0}^2 + n \cdot \bar{y}/\sigma^2}{1/\sigma_{\mu 0}^2 + n/\sigma^2}, \frac{1}{1/\sigma_{\mu 0}^2 + n/\sigma^2}\right)$$

- What happens when n is big?

μ known, σ unknown

Steps for Bayesian inference about σ are same as before but easiest set-up is based on $1/\sigma^2$, which is called the precision:

$$p\left(1/\sigma^2 | \mathbf{y}, \mu\right) = \frac{p\left(1/\sigma^2\right) p(\mathbf{y} | \mu, \sigma)}{p(\mathbf{y})} \propto p\left(1/\sigma^2\right) p(\mathbf{y} | \mu, \sigma)$$

The Gamma distribution for the precision is a conjugate prior :

► If $1/\sigma^2 \sim \text{Gamma}(\nu_0/2, \nu_0/2 \cdot \sigma_0^2)$, then

$$1/\sigma^2 | \mathbf{y}, \mu \sim \text{Gamma}\left(\nu_n/2, \nu_n/2 \cdot \sigma_n^2\right)$$

with

$$\nu_n = \nu_0 + n$$

$$\sigma_n^2 = 1/\nu_n \left(\nu_0 \sigma_0^2 + n s_n^2\{\mu\} \right)$$

$$s_n^2\{\mu\} = 1/n \sum (y_i - \mu)^2$$

μ and σ unknown

Use same priors as before such that we know what these full conditionals are:

$$\mu \sim N(\mu_0, \sigma_{\mu 0}^2)$$

$$\mu | \mathbf{y}, \sigma^2 \sim N\left(\frac{\mu_0 / \sigma_{\mu 0}^2 + n \cdot \bar{y} / \sigma^2}{1 / \sigma_{\mu 0}^2 + n / \sigma^2}, \frac{1}{1 / \sigma_{\mu 0}^2 + n / \sigma^2}\right)$$

$$1 / \sigma^2 \sim \text{Gamma}\left(\nu_0 / 2, \nu_0 / 2 \cdot \sigma_0^2\right)$$

$$1 / \sigma^2 | \mathbf{y}, \mu \sim \text{Gamma}\left(\nu_n / 2, \nu_n / 2 \cdot \sigma_n^2\right)$$

with

$$\nu_n = \nu_0 + n; \sigma_n^2 = 1 / \nu_n (\nu_0 \sigma_0^2 + n s_n^2 \{\mu\})$$

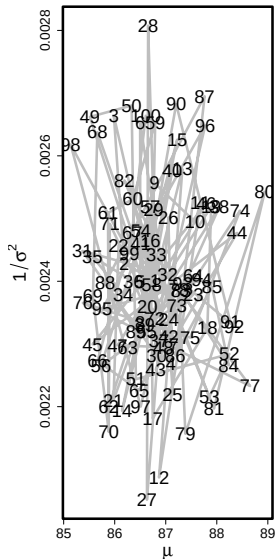
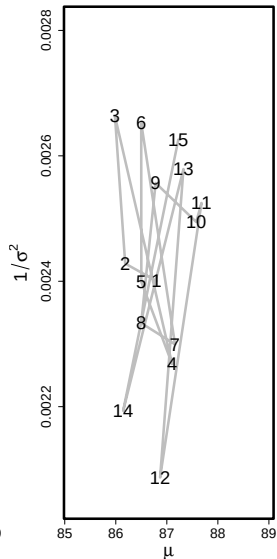
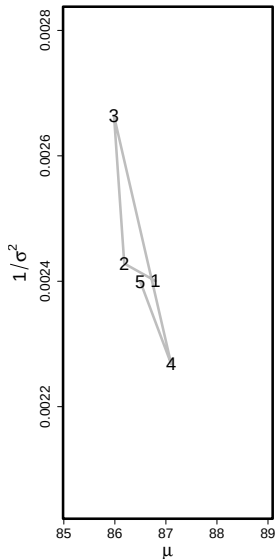
$$n \cdot s_n^2 \{\mu\} = \sum (y_i - \mu)^2 = (n - 1) s^2 \{y\} + n (\bar{y} - \mu)^2 \text{ (Hoff p.94)}$$

Gibbs Sampling

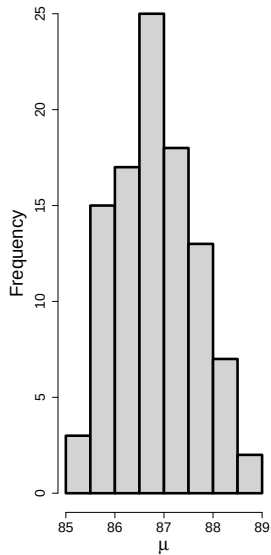
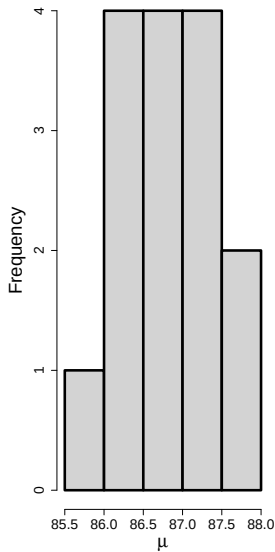
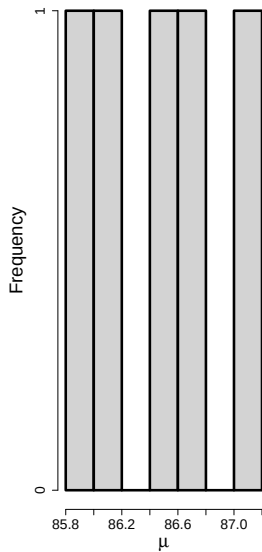
- ▶ Calculate/define values for $\bar{y}$, hyperparameters (μ_0 etc) and corresponding values of posterior distribution parameters
- ▶ Choose some starting point $\mu^{(1)}$ and $\sigma^{(1)}$
- ▶ Sample $\mu^{(2)}$ from full conditional (using $\sigma^{(1)}$)
- ▶ Sample $\sigma^{(2)}$ from full conditional (using $\mu^{(2)}$)
- ▶ etc etc etc

Next slide: data-driven values for hyperparameters, starting initial values at means

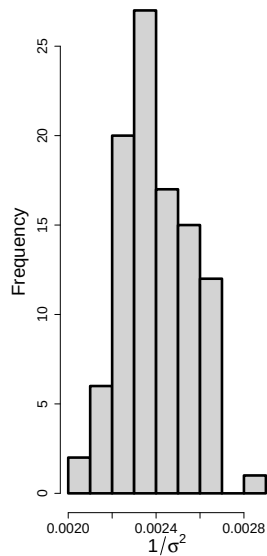
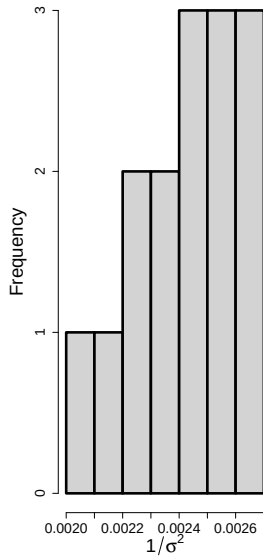
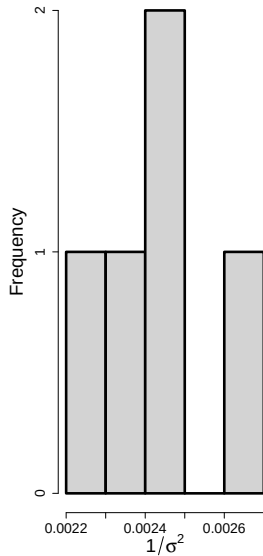
Gibbs Sampling (first 5, 10, 100 iterations)



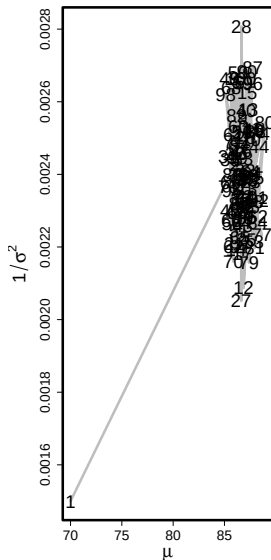
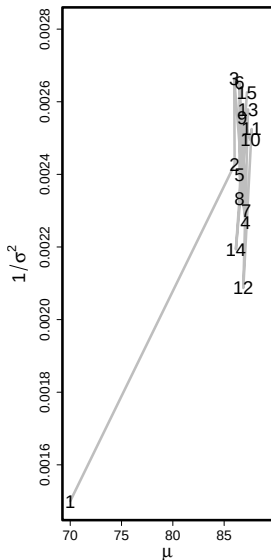
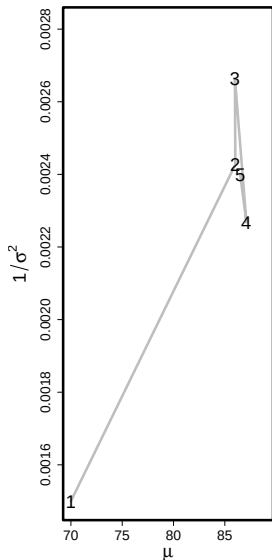
Histograms of μ (5, 15, 100 samples)



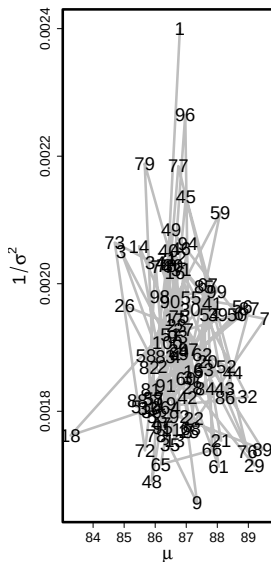
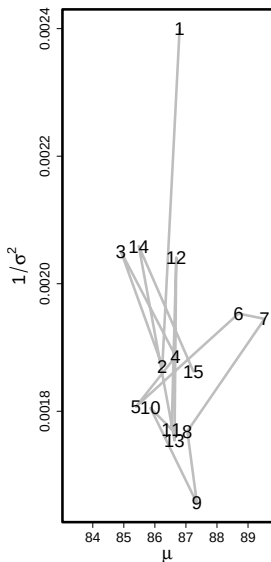
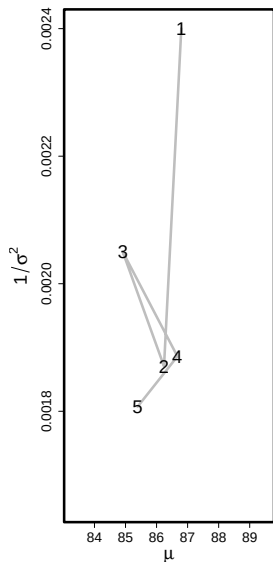
Histograms of $1/\sigma^2$ (5, 15, 100 samples)



What about a different starting point



What about different hyperparameters



Gibbs Sampling

- ▶ When doing inference for μ when σ is known, and inference for σ when μ is known, conjugate priors can be used and the posteriors are available in closed form.
- ▶ When doing inference for both simultaneously, we can still use Bayes' rule but finding closed-form expressions for the (joint) posteriors is hard/impossible, so can use sampling.
- ▶ Gibbs sampling is one of such methods, where (subsets of) parameters are sampled sequentially from their full conditional distributions (given by conditioning on the data as well as all other parameters).

MCMC

MCMC

The Gibbs sampling algorithm is a Markov Chain Monte Carlo (MCMC) algorithm.

The **Markov Chain** part

- ▶ Let $\phi^{(s)} = (\mu^{(s)}, \sigma^{(s)})$ be the s -th draw of parameters.
- ▶ $\phi^{(s)}$ depends on $\phi^{(s-1)}, \phi^{(s-2)}, \dots, \phi^{(1)}$ only through $\phi^{(s-1)}$
- ▶ This is called the Markov property, and so the sequence is called a Markov chain
- ▶ Each sample $\phi^{(s)}$ is drawn such that eventually, for some large s , $\phi^{(s)}$ is a draw from the target distribution, which in this example is the posterior distribution $(\mu, \sigma) | \mathbf{y}$.

The **Monte Carlo** part

- ▶ We approximate quantities of interest, e.g. $E(\mu | \mathbf{y})$, using resulting samples.

MCMC: beyond Gibbs

- ▶ In Gibbs Sampling we used conjugate priors to write down the full conditionals
- ▶ Sometimes conjugate priors are not available or are unsuitable.
- ▶ For more complex models, we often cannot find closed form solutions for the posterior

Any algorithm for obtaining samples from a target distribution (e.g. $p(\theta|y)$) whereby the s -th sample $\theta^{(s)}$ depends on $\theta^{(s-1)}$ is called an MCMC algorithm.

MCMC: intuition

- ▶ Suppose we already have $\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(s)}$ and we have another θ . Should we add θ to our samples?
- ▶ Want to compare $p(\theta|y)$ to $p(\theta^{(s)}|y)$
- ▶ But we don't need to compute the posteriors

$$\frac{p(\theta|y)}{p(\theta^{(s)}|y)} = \frac{p(y|\theta)p(\theta)p(y)}{p(y)p(y|\theta^{(s)})p(\theta^{(s)})} = \frac{p(y|\theta)p(\theta)}{p(y|\theta^{(s)})p(\theta^{(s)})}$$

- ▶ If $p(\theta|y) > p(\theta^{(s)}|y)$ we set $\theta = \theta^{(s+1)}$.
- ▶ If $r = p(\theta|y)/p(\theta^{(s)}|y) < 1$ we expect to have θ appear r times as often as $\theta^{(s)}$ so we accept θ with probability r .

The Metropolis algorithm

1. Propose a new θ^* which is a draw from a symmetric distribution around $\theta^{(s)}$, call it $J(\theta^*|\theta^{(s)})$ (symmetry means $J_t(\theta_a|\theta_b) = J_t(\theta_b|\theta_a)$)
2. Calculate the ratio

$$r = \frac{p(\theta^*|\mathbf{y})}{p(\theta^{(s)}|\mathbf{y})} = \frac{p(\theta^*)p(\mathbf{y}|\theta^*)}{p(\theta^{(s)})p(\mathbf{y}|\theta^{(s)})}$$

3.
 - ▶ If $r > 1$, set $\theta^{(s+1)} = \theta^*$.
 - ▶ If $r < 1$, set $\theta^{(s+1)} = \theta^*$ with probability r and $\theta^{(s+1)} = \theta^s$ with probability $1 - r$.

The Metropolis-Hasting algorithm

Extension to non-symmetric proposal distributions. To correct for the asymmetry, the ratio r is replaced by a ratio of ratios:

$$r = \frac{p(\theta^*|y) / J_{s+1}(\theta^*|\theta^s)}{p(\theta^s|y) / J_{s+1}(\theta^s|\theta^*)}$$

- ▶ Metropolis a special case of MH
- ▶ Gibbs a special case of MH with $r = ?$

Does it work? (in theory)

Supplementary details: Hoff Chapter 10

- ▶ For most problems, we can construct an MH algorithm that generates an *irreducible*, *aperiodic* and *recurrent* Markov chain, that converges to a stationary distribution which is the posterior distribution of interest.
- ▶ What that means is: Even though samples are obtained sequentially (and depend on the previous sample), eventually, the s -th draw can be considered to be a random draw from the target distribution, i.e.:

Irreducible, aperiodic and recurrent

What is an irreducible, aperiodic and recurrent Markov chain?

- ▶ Irreducible: able to move from one value of θ with $p(\theta|\mathbf{y}) > 0$ to another with non-zero probability mass
- ▶ Aperiodic: no periodic values for θ with $p(\theta|\mathbf{y}) > 0$
- ▶ Recurrent: if we start at θ with $p(\theta|\mathbf{y}) > 0$, we are guaranteed to return to it.

Once you are sampling from the stationary distribution, you are always sampling from the stationary distribution.

Proposal distributions

In most problems it is not hard to construct MH algorithms that generate Markov chains that are irreducible, aperiodic and recurrent.

What should we choose for our proposal distribution $J(\theta^*|\theta^{(s)})$?

A normal proposal distribution centered at the current value works. This is called **Random Walk Metropolis**

- ▶ take our current position and add some noise

$$J(\theta^*|\theta^{(s)}) \sim N(\theta^{(s)}, \sigma^2)$$

i.e.

$$\theta^* = \theta^{(s)} + \epsilon^*$$

Issues with random walk Metropolis:

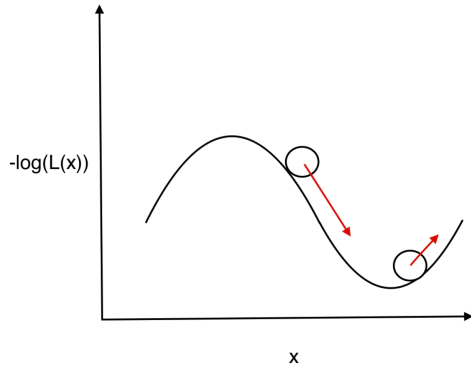
- ▶ slow
- ▶ highly correlated samples
- ▶ breaks down in high dimensions

Proposal distributions

Common alternative is the class of proposal distributions that lead to **Hamiltonian Monte Carlo**:

- ▶ Physical analogy of a particle moving around some parameter space i.e. treat the Markov chain like a physical particle.
- ▶ The 'energy' of particle (potential and kinetic) is dependent on its position and momentum. We know position, we generate momentum.
- ▶ Treat the negative log density like a physical potential.
- ▶ Proposal distributions makes use of gradient information of the log of the posterior in order to move more quickly toward regions of high probability.
- ▶ Give the particle a random shove instead of a random step.
- ▶ When it fails, it fails badly (cf RWMH, which fails silently)

HMC



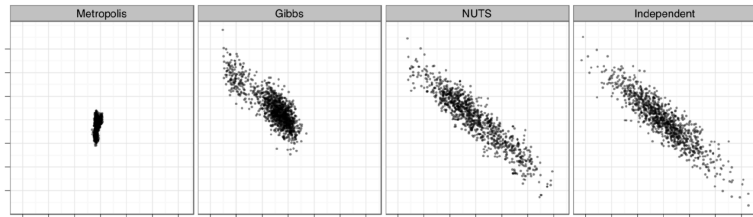


Figure from Hoffman and Gelman

Where are we at

- ▶ For most problems, we can construct an MCMC algorithm that generates an irreducible, aperiodic and recurrent Markov chain, that converges to a stationary distribution which is the posterior distribution of interest.
- ▶ Then, eventually, an MCMC sample is a draw from its target distribution, hence MCMC algorithms can be used to generate samples from posterior distributions. . .

Can we just use all the samples?

Does it work (in practice?)

MCMC samples are NOT independent draws from a target distribution:

- ▶ The first draw is set by the user and thus not a random draw from the target distribution.
- ▶ Draw $s + 1$ is correlated with draw s

We can use samples from an MCMC algorithm to do inference but ONLY IF we have “waited long enough” for those samples to be representative of the distribution of interest.

Need to (try and) check:

- ▶ That the MCMC chain has converged away from the initial values into the target (stationary) distribution.
- ▶ That the chain has generated a representative sample

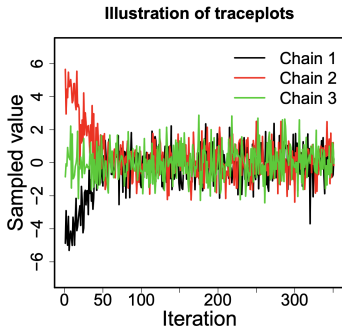
MCMC diagnostic tools

- ▶ Trace plots
- ▶ Effective sample size
- ▶ $\hat{R}$

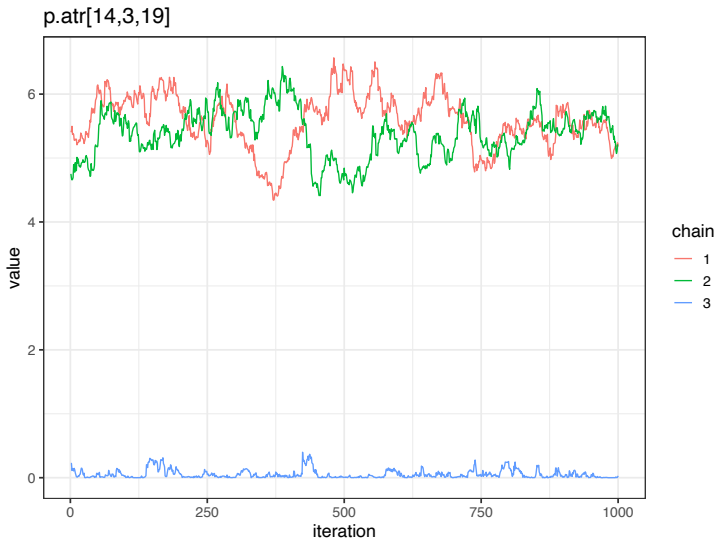
Trace plots

Trace plots: can be used to detect **burn-in** and **mixing**.

- ▶ Initial MCMC samples may not be representative of the posterior distribution.
- ▶ The initial phase of an MCMC chain is called the burn-in phase, during which the chain converges towards the target distribution.
- ▶ Samples from the burn-in period should be discarded.
- ▶ Chains should be 'mixing' well

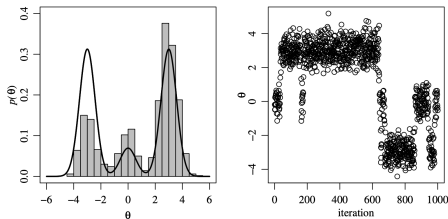


A bad trace plot



Effective sample size

S_{eff} = the number of independent MC samples that would give the same precision for the mean as the MCMC samples, estimated by $\bar{\theta} = 1/S \sum_s \theta^{(s)}$, as obtained with the MCMC sample of size S .



Hoff Figure 6.5, $S_{eff} = 18.4$ ($S = 1,000$).

Effective sample size

- ▶ The precision (standard error) of the mean with MC samples is just

$$\text{Var}(\bar{\theta})_{MC} = \frac{\text{Var}(\theta)}{S}$$

- ▶ But for MCMC, the samples S are not independent, and they are, in general, positively correlated
- ▶ So in general $\text{Var}(\bar{\theta})_{MCMC} > \text{Var}(\bar{\theta})_{MC}$
- ▶ The effective sample size is calculated as

$$S_{eff} = \frac{N}{1 + 2 \sum_t \rho_t}$$

where ρ_t is the autocorrelation at lag t . S_{eff} can be interpreted as the number of independent Monte Carlo samples necessary to give the same precision as the MCMC samples.

Potential scale reduction factor $\hat{R}$

The Gelman-Rubin convergence diagnostic statistic $\hat{R}$ (Gelman & Rubin 1992), or potential scale reduction factor/shrink factor, is based on a comparison between the average variance of samples within each chain to the variance of the pooled samples across chains

- ▶ If chains have mixed well, then $\hat{R}$ is close to 1
- ▶ Rules of thumb: Aim for $\hat{R} < 1.05$, and S_{eff} greater than 100.

More when we start fitting in R

Running MCMC in R

Running MCMC in R

Good news:

- ▶ lots of standard software to run MCMC so that we (usually) don't have to code it ourselves
- ▶ easy to implement (m)any Bayesian models as long as you can write down the model specification
- ▶ The user does not need to write the MCMC algorithm
- ▶ But you have to check MCMC output for convergence/mixing (and understand why it might not be working)

BUGS/JAGS

- ▶ BUGS: Bayesian Inference Using Gibbs Sampling (includes WinBUGS, OpenBUGS)
- ▶ JAGS: Just Another Gibbs Sampler (BUGS but not platform dependent).
- ▶ Very common from late 2000s, still main tool used in many applied fields (particularly WinBUGS in epidemiological research where spatial modeling is common)
- ▶ Run models by writing a 'model' (as text) which specifies your likelihood and priors.

Stan

- ▶ Increasingly common to use on a wide range of estimation problems
- ▶ Estimates using a version of HMC (No-U-Turn-Sampler)
- ▶ Stan consists of a language for defining probabilistic models, a compiler to transform Stan code into C++, and math and algorithm libraries that help run models
- ▶ We will be running Stan through R using RStan; can also run in CMD or Python

Running Stan: overview

- ▶ Write a Stan program (in a .stan file), specifying data, parameters, model, and potentially other things
- ▶ Input data, run model
- ▶ Output: an approximate sample from the posterior distribution, summaries of the run which can help us diagnose problems.

Writing a model in Stan

Like C++, in Stan:

- ▶ Variables need to be declared and strongly typed
- ▶ Each statement must end with a semi-colon

A Stan program is divided into coding blocks. Essential:

- ▶ data
- ▶ parameters
- ▶ model

Optional:

- ▶ transformed parameters
- ▶ generated quantities
- ▶ ...

A Stan file

save as `whatever.stan`

```
data {  
  Declare the data that will be given as an input.  
}  
  
parameters {  
  Declare the parameters we want to sample.  
}  
  
model {  
  Compute the log joint distribution.  
}
```

Back to Kid's test scores

Recall model set up

$$y_i | \mu, \sigma \sim N(\mu, \sigma^2), \quad (\text{independent})$$

$$\mu \sim N(\mu_0, \sigma_{\mu 0}^2)$$

$$1/\sigma^2 \sim \text{Gamma}(\nu_0/2, \nu_0/2 \cdot \sigma_{y0}^2)$$


```

data {
  int<lower=0> N;           // number of kids
  vector[N] y;            // scores
  real mu0;               // mean of mu prior
  real<lower=0> sigma0;    // variance of mu prior
  real<lower=0> nu0;       // shape of precision prior
}

parameters {
  real mu;
  real<lower=0> tau;
}

transformed parameters {
  real<lower=0> sigma = sqrt(1/tau);
}

model {
  //priors
  mu ~ normal(mu0, sigma0);
  tau ~ gamma(nu0/2, nu0/2*sigma0^2);

  //target += normal_lpdf(y | mu, sigma);
  //equivalent:
  y ~ normal(mu, sigma);
}

```

Kid's example

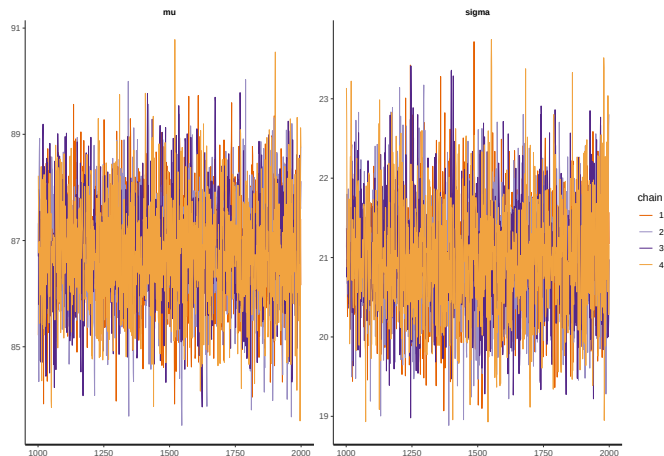
```
library(rstan)
data <- list(y = y, N = length(y),
             mu0 = mu0,
             sigma0 = sigma.0,
             nu0 = 1)
fit <- stan(file = "kids.stan",
            data = data)
```

fit

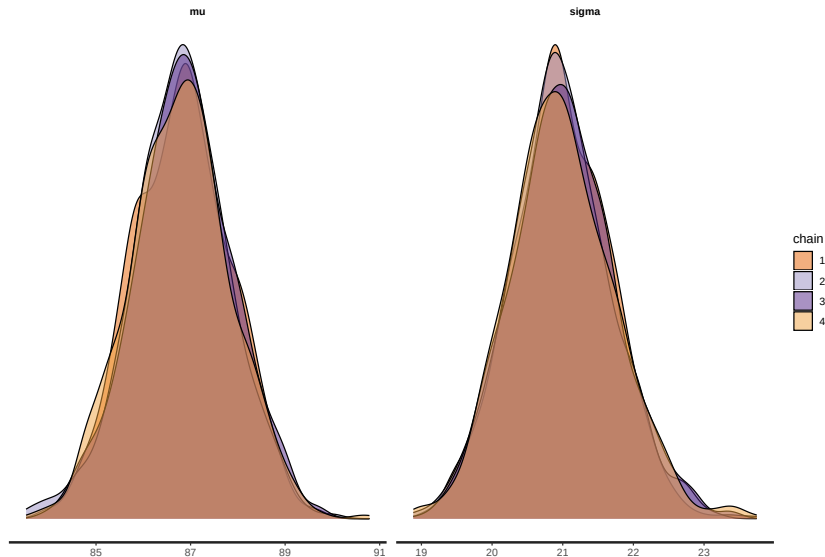
```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##               mean se_mean   sd      2.5%      25%      50%      75%      97.5% n_eff
## mu           86.82    0.02 1.03    84.76    86.12    86.83    87.48    88.86  3584
## tau           0.00    0.00 0.00     0.00     0.00     0.00     0.00     0.00  3791
## sigma        20.99    0.01 0.71    19.65    20.51    20.96    21.46    22.46  3756
## lp__        -1541.62    0.02 1.02 -1544.43 -1542.04 -1541.29 -1540.88 -1540.62 1757
##               Rhat
## mu              1
## tau              1
## sigma            1
## lp__             1
##
## Samples were drawn using NUTS(diag_e) at Fri Jan 30 08:54:10 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

Some graphical checks

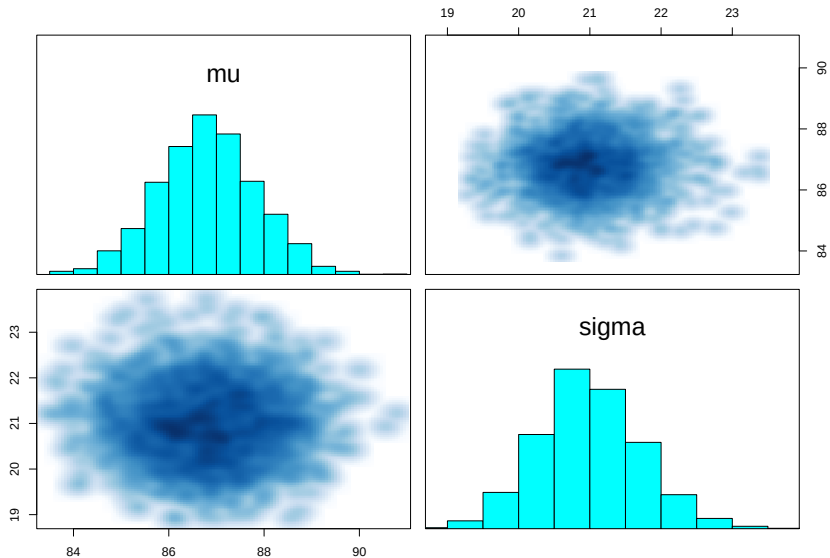
```
pars = c("mu", "sigma")  
traceplot(fit, pars = pars)
```



```
stan_dens(fit, separate_chains = TRUE, pars = pars)
```



```
pairs(fit, pars = pars)
```



Summary and expectations

- ▶ MCMC one general class of algorithms for estimating posterior distributions
- ▶ Note that we can never be sure that a chain has converged but at least we can detect lack of convergence.

I expect you to:

- ▶ Understand main differences between different samplers discussed
- ▶ Run models in R and interpret output
- ▶ Diagnose model problems and think about how to fix them/
fix them